get stuck in local minima and plateaus. And, even worse, it's taking a slower way to get there.

On the plus side, it also has some advantages: first and foremost, we can release the constraint about the cost function being differentiable, and actually even ignore the definition of the function, as long as we have a way to compute it[3]. This is especially useful with functions like the edge crossing of an embedding, a step function which is not differentiable in the points where its value abruptly changes, and has a derivative identically equal to zero elsewhere.

The obvious downside, as so well shown in figure 17.2, is that the random optimization is going to require many more steps than gradient descent, because it will take long detours; even worse, since it will not take the direction where the function decreases faster, it will likely end up in a different spot than gradient descent: it's impossible to predict where, and as such whether it will find a better or worse final value. To cope with this, the only strategy is perseverance: increasing the number of runs and storing the best result across them is likely to bring to a satisfying result (the more runs, the better the expected solution).

Finally, there are issues faced in gradient descent that not even the randomized method can shake off.

For instance, while it's possible that the random optimization algorithm get out of a local minimum, it's not guaranteed: since both algorithms are greedy[4], they only move from current position when they find a domain point to which corresponds a lower cost, it's possible that the range of the random steps is not wide enough to get it out of a pit. You